

Image Analysis, Assignment 4

1 Color correction of images

In this task you will investigate ways of changing, correcting and enhancing images, and work with simple algorithms and color spaces described in the course.

1) In the file `abbey_badcolor.jpg` an image is given. There is an undesirable color cast in the image. Implement the basic "gray world" assumption to automatically white balance the image, i.e. with a simple scaling of each color channel.

2) Modify the basic "gray world"-method in some way to improve the result, i.e. experiment with pre-processing and post-processing (smoothing, intensity changes, use patches) to make it work better.

3) Try the learned method for color correction that was described in the lecture. The method is described and implemented here:

https://github.com/mahmoudnafifi/WB_sRGB/

Download the code and run either the Matlab or Python version to correct the image `abbey_badcolor.jpg`.

4) In the file `abbey_correct.jpg` the correctly white balanced image is given. For your three outputs, from the three methods, calculate both the *PSNR* and *SSIM* metrics, compared to the correct image. Also use the perceptual error metric $\mathcal{FLIP}$ to also calculate the $\mathcal{FLIP}$ -error for the three output images.

Matlab-specific instructions

After you have downloaded the learned color correction method of Afifi at https://github.com/mahmoudnafifi/WB_sRGB/, look at the script `demo.m` in the Matlab folder `WB_sRGB_Matlab` in the repository, to see how it works. For PSNR and SSIM metrics you may use the built-in Matlab implementations, but make sure that you understand roughly how these metrics are calculated. In the task folder for the handin you will find a folder with the Matlab implementation of the perceptual metric $\mathcal{FLIP}$. You can calculate the error for two images with the function: `err = computeFLIP(im_ref,im_test);`

Python-specific instructions

There is an outline of a script that solves the task given in `task1.py`. After you have downloaded the learned color correction method of Afifi at https://github.com/mahmoudnafifi/WB_sRGB/, look at the script `demo.py` in the Python folder `WB_sRGB_Python` in the repository, to see how it works.

You may want to set `upgraded_model = 1`. Also you need to install the `opencv` module to run the demo script. For PSNR and SSIM metrics you may use Python implementations available from e.g. the module `scikit-image`, but make sure that you understand roughly how these metrics are calculated. In the task folder for the handin you will find a folder with the Python implementation of the perceptual metric FLIP. You can calculate the error for two images with the function: `err = computeFLIP(im_ref, im_test)`. See the main script in `flip.py` for an example on how to use it.

For the report: Describe your implemented methods and supply both your code (Matlab/Python code) and the three resulting output images. Also provide the 3×3 -table with the computed errors.

2 Segmentation with Graph Cuts

You are given the task of segmenting out two heart chambers in an image by Graph-Cuts. In `heart_data.mat` you are given two vectors, with a number of intensities that have been observed inside the two chambers (`chamber_values`) and a number of intensities observed in the background (`background_values`). You are also given the image of Figure 1, denoted by f below.

We will in this task use an available implementation for solving the graph cut problem using max flow. Study the material in the lectures and also read the chapter on Graph Cut segmentation in Szelisky "Computer Vision: Algorithms and Applications", and modify the script according to the following tasks:

1) Assume that the pixel intensities in the two classes are generated by two different Gaussian distributions. In other words, for pixel i , the likelihoods of observing intensity f_i , $P(f_i | \text{chamber class})$ and $P(f_i | \text{background class})$, respectively, are Gaussians. Recall, a Gaussian is given by $P(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$. Estimate the means and the standard deviations (μ and σ) for the chamber class and the background class, respectively, using the provided ground truth values in the data file.

2) Given the theoretical assumptions on the distributions (Gaussian), what is the negative log likelihoods for the two classes (it should be a simple expression, derived analytically, and you can ignore the term arising from the normalizing constant in the Gaussian)? Use the statistical Mumford-Shah functional formulation, i.e. instead of using the formulation based on

$$E_0(\Gamma) = \int_{R_1} (a_1 - f)^2 dx dy + \int_{R_2} (a_2 - f)^2 dx dy + \nu |\Gamma|, \quad (1)$$

use a data-term consisting of the negative log likelihoods for the two classes, in the sense of

$$E_0(\Gamma) = \int_{R_1} -\log P(f|\text{chamber class})dxdy + \int_{R_2} -\log P(f|\text{background class})dxdy + \nu|\Gamma| \quad (2)$$

Then construct a graph of the heart image with such data terms, and solve the corresponding discretized problem via max-flow/min-cut. Experiment with the prior/regularization weight ν , in order to obtain a reasonable segmentation. (Note: it is hard to get a perfect segmentation.)

3) Optional. Fine tune the segmentation by adding ground truth. This can be done by changing the data-term to ∞ (or a large number) for the pixels you know which region they belong to. This will force certain pixels to belong to a particular class. Such pixels can be obtained by manual inspection of the image.

Notes: Both 2) and 3) can be seen as adjustments a medical doctor could perform. In 2) a slider could be added to give interactive response to different weights. Then a pen tool can be used to introduce the ground truth of 3).

Matlab-specific instructions

In `task2.m` there is a script that shows how you can set up a graph and use the Matlab function `maxflow.m` to solve the min-cut problem. Change the script so that the weights are set according to the given task, using the negative log likelihood formulation, and with the estimated means and standard deviations.

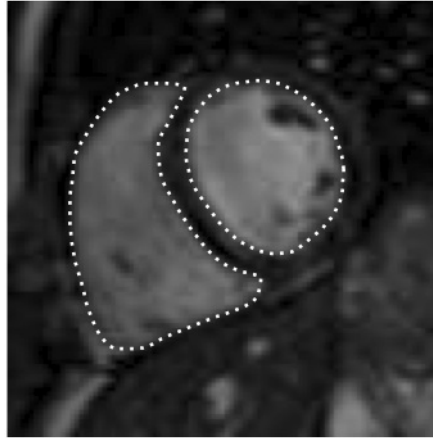


Figure 1: The data given in the task. A slice of a heart imaged by a MRI camera. This view is known as the "short-axis view" taken in a direction where the left and the right heart chambers are visible at the same time. The two chambers are shown inside the dotted lines, the left chamber is to the right in the image and vice versa.

Python-specific instructions

In `task2.py` there is example code for setting up a graph and running the Scipy implementation of maximum flow. It uses a special sparse matrix format. There is also code for generating neighborhood indices for a given size of input image (`edges4/8connected`, don't change these functions!). Please modify the code (calculating the means and standard deviations of the given data, and setting up the weights for both the regularization term and the data term for graph cut segmentation, based on the statistical likelihood model).

For the report: supply your code (Matlab/Python code), the estimated means and standard deviations, and a printout of the results of using your algorithm, i.e. supply input data (as an image) and result after applying your segmentation algorithm (also as an image).

3 Computer Vision

Assume that the camera matrices for two projections are

$$P_1 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

and

$$P_2 = \begin{pmatrix} 1 & 2 & 3 & 3 \\ 3 & 2 & 0 & -2 \\ 1 & 1 & 2 & 0 \end{pmatrix}.$$

The so-called fundamental matrix is then

$$F = \begin{pmatrix} 2 & 2 & 4 \\ 3 & 3 & 6 \\ -11 & -10 & -6 \end{pmatrix}.$$

The following three points are detected in image 1:

$$a_1 = (2, 8), \quad a_2 = (-3, 0), \quad a_3 = (1, 16).$$

In image 2 the following three points are detected:

$$b_1 = (3, 1), \quad b_2 = (9, 3), \quad b_3 = (0, -7).$$

Which points can be in correspondence? (Check all possible combinations of points in image 1 and image 2, nine in total).

For the report: Provide your calculations, your answer and your motivation.

4 OCR system construction and system testing

Here you will finish the work on your OCR system. From the three previous assignments you have constructed

- `im2segment(Im)` - a segmentation algorithm that takes an image to a number of segments.
- `segment2features(S)` - an algorithm for calculating a feature vector x from a segment S .
- `features2class(x,classification_data)` - an algorithm for classifying a feature vector x as class y using machine learning.

The data files for this assignment contain both new test examples as well as a benchmark script. Run the benchmark script on all of the five datasets (short1, short2, home1, home2, home3). Write down the results before modifying your code. Then try improving your code to make it as good as possible.

- Test your system version 1 from Assignment 3 on the five datasets (short1, short2, home1, home2, home3).
- Try to change your system and produce a version 2 that works better on the five datasets.

Matlab-specific instructions

Try running `inl4_test_and_benchmark` on all of the five datasets (short1, short2, home1, home2, home3).

Python-specific instructions

In `assignment4.py`, substitute the bad implementations of the three functions with your own. Then test your system on the five datasets, using e.g. the main script in `assignment4.py`.

Print the overall hitrate for version 1 on the five datasets in the report and describe with your own words the results in your report. If it went well, describe why. If it didn't go well describe why. Do the same for version 2 of your system. Describe what modifications you did in version 2 and why. You should aim at hitrates higher than 50% for all datasets. If you don't reach this level, describe what attempts you have made to increase the hitrate for version 2. Also include the Matlab/Python code of your final system.